

EECS 182 Deep Neural Networks
Spring 2025 Anant Sahai

Homework 7

This homework is due on Mar 30, at 10:59PM.

1. RNNs for Last Name Classification

Please follow the instructions in [this notebook](#). You will train a neural network to predict the probable language of origin for a given last name / family name in Latin alphabets.

- (a) Although the neural network you have trained is intended to predict the language of origin for a given last name, it could potentially be misused. **In what ways do you think this could be problematic in real-world applications?**

2. Auto-encoder : Learning without Labels

So far, with supervised learning algorithms we have used labelled datasets $\mathcal{D} = \{\mathbf{X}, \mathbf{y}\}$ to learn a mapping f_θ from input x to labels y . In this problem, we now consider algorithms for *unsupervised learning*, where we are given only inputs x , but no labels y . At a high-level, unsupervised learning algorithms leverage some structure in the dataset to define proxy tasks, and learn *representations* that are useful for solving downstream tasks.

Autoencoders present a family of such algorithms, where we consider the problem of learning a function $f_\theta : \mathbb{R}^m \rightarrow \mathbb{R}^k$ from input x to a *intermediate representation* z of x (usually $k \ll m$). For autoencoders, we use reconstructing the original signal as a proxy task, i.e. representations are more likely to be useful for downstream tasks if they are low-dimensional but encode sufficient information to approximately reconstruct the input. Broadly, autoencoder architectures have two modules:

- An encoder $f_\theta : \mathbb{R}^m \rightarrow \mathbb{R}^k$ that maps input x to a representation z .
- A decoder $g_\phi : \mathbb{R}^k \rightarrow \mathbb{R}^m$ that maps representation z to a reconstruction $\hat{x}$ of x .

In such architectures, the parameters (θ, ϕ) are learnable, and trained with the learning objective of minimizing the reconstruction error $\ell(\hat{x}_i, x_i) = \|x - \hat{x}\|_2^2$ using gradient descent.

$$\theta_{\text{enc}}, \phi_{\text{dec}} = \underset{\Theta, \Phi}{\operatorname{argmin}} \frac{1}{N} \sum_{i=1}^N \ell(\hat{x}_i, x_i)$$

$$\hat{x} = g_\phi(f_\theta(x))$$

Note that above optimization problem does not require labels y . In practice however, we would want to use the *pretrained* models to solve the *downstream* task at hand, e.g. classifying MNIST digits. *Linear Probing* is one such approach, where we fix the encoder weights, and learn a simple linear classifier over the features $z = \text{encoder}(x)$.

(a) Designing AutoEncoders

Please follow the instructions in [this notebook](#). You will train autoencoders, denoising autoencoders, and masked autoencoders on a synthetic dataset and the MNIST dataset. Once you finished with the notebook,

- Answer the following questions in your submission of the written assignment:

- (i) **Show your visualization** of the vanilla autoencoder with different latent representation sizes.
- (ii) Based on your previous visualizations, answer this question: **How does changing the latent representation size of the autoencoder affect the model's performance in terms of reconstruction accuracy and linear probe accuracy? Why?**

(b) PCA & AutoEncoders

In the case where the encoder f_θ, g_ϕ are linear functions, the model is termed as a *linear autoencoder*. In particular, assume that we have data $x_i \in \mathbb{R}^m$ and consider two weight matrices: an encoder $W_1 \in \mathbb{R}^{k \times m}$ and decoder $W_2 \in \mathbb{R}^{m \times k}$ (with $k < m$). Then, a linear autoencoder learns a low-dimensional embedding of the data $\mathbf{X} \in \mathbb{R}^{m \times n}$ (which we assume is zero-centered without loss of generality) by minimizing the objective,

$$\mathcal{L}(W_1, W_2; \mathbf{X}) = \|\mathbf{X} - W_2 W_1 \mathbf{X}\|_F^2 \quad (1)$$

We will assume $\sigma_1^2 > \dots > \sigma_k^2 > 0$ are the k largest eigenvalues of $\mathbf{X}\mathbf{X}^\top$. The assumption that the $\sigma_1, \dots, \sigma_k$ are positive and distinct ensures identifiability of the principal components, and is common in this setting. Therefore, the top- k eigenvalues of $\mathbf{X}$ are $S = \text{diag}(\sigma_1, \dots, \sigma_k)$, with corresponding eigenvectors are the columns of $\mathbf{U}_k \in \mathbb{R}^{m \times k}$. A well-established result from ¹ shows that principal components are the unique optimal solution to linear autoencoders (up to sign changes to the projection directions). In this subpart, we take some steps towards proving this result.

- (i) **Write out the first order optimality conditions that the minima of Eq. 1 would satisfy.**
- (ii) **Show that the principal components $\mathbf{U}_k$ satisfy the optimality conditions outlined in (i).**

3. Self-supervised Linear Autoencoders

We consider linear models consisting of two weight matrices: an encoder $W_1 \in \mathbb{R}^{k \times m}$ and decoder $W_2 \in \mathbb{R}^{m \times k}$ (assume $1 < k < m$). The traditional autoencoder model learns a low-dimensional embedding of the n points of training data $\mathbf{X} \in \mathbb{R}^{m \times n}$ by minimizing the objective,

$$\mathcal{L}(W_1, W_2; \mathbf{X}) = \frac{1}{n} \|\mathbf{X} - W_2 W_1 \mathbf{X}\|_F^2 \quad (2)$$

We will assume $\sigma_1^2 > \dots > \sigma_k^2 > \sigma_{k+1}^2 \geq 0$ are the $k+1$ largest eigenvalues of $\frac{1}{n} \mathbf{X}\mathbf{X}^\top$. The assumption that the $\sigma_1, \dots, \sigma_k$ are positive and distinct ensures identifiability of the principal components.

Consider an ℓ_2 -regularized linear autoencoder where the objective is:

$$\mathcal{L}_\lambda(W_1, W_2; \mathbf{X}) = \frac{1}{n} \|\mathbf{X} - W_2 W_1 \mathbf{X}\|_F^2 + \lambda \|W_1\|_F^2 + \lambda \|W_2\|_F^2. \quad (3)$$

where $\|\cdot\|_F^2$ represents the Frobenius norm squared of the matrix (i.e. sum of squares of the entries).

- (a) You want to use SGD-style training in PyTorch (involving the training points one at a time) and self-supervision to find W_1 and W_2 which optimize (3) by treating the problem as a neural net being trained in a supervised fashion. **Answer the following questions and briefly explain your choice:**
 - (i) **How many linear layers do you need?**

☐ 0

☐ 1

¹Baldi, Pierre, and Kurt Hornik. "Neural networks and principal component analysis: Learning from examples without local minima" (1989)

☐ 2☐ 3(ii) **What is the loss function that you will be using?**☐ `nn.L1Loss`☐ `nn.MSELoss`☐ `nn.CrossEntropyLoss`(iii) **Which of the following would you need to optimize (3) exactly as it is written? (Select all that are needed)**☐ Weight Decay☐ Dropout☐ Layer Norm☐ Batch Norm☐ SGD optimizer(b) **Do you think that the solution to (3) when we use a small nonzero λ has an inductive bias towards finding a W_2 matrix with approximately orthonormal columns? Argue why or why not?**

(Hint: Think about the SVDs of $W_1 = U_1 \Sigma_1 V_1^\top$ and $W_2 = U_2 \Sigma_2 V_2^\top$. You can assume that if a $k \times m$ or $m \times k$ matrix has all k of its nonzero singular values being 1, then it must have orthonormal rows or columns. Remember that the Frobenius norm squared of a matrix is just the sum of the squares of its singular values. Further think about the minimizer of $\frac{1}{\sigma^2} + \sigma^2$. Is it unique?)

4. SSM Convolution Kernel

Background and Setup: Consider a discrete-time State-Space Model (SSM) of the form

$$\begin{aligned}x_{k+1} &= Ax_k + Bu_k, \\ y_k &= Cx_k + Du_k,\end{aligned}$$

(a) **Convolution Kernel and the Output Equation.** Given that the sequence length is L (input: $(u_0, \dots, u_L)$, output: $(y_0, \dots, y_L)$) and assume $x_0 = 0$, **show** that the output y_k can be expressed as a *convolution* of the input sequence $\{u_\ell\}_0^L$ with a kernel $K = \{K_\ell\}_0^L$:

$$y_k = \sum_{\ell=0}^L K_\ell u_{k-\ell},$$

where any $u_{\leq 0}$ with a negative index is set to 0 (zero-padding). Also, **find** K .

(b) **Concrete Examples.**

i. *Scalar Case:* Let $n = 1$, and set $A = \alpha$, $B = \beta$, $C = \gamma$, $D = \delta$. Use $\alpha = 0.8, \beta = 1, \gamma = 1.5$ and compute the kernel up to $L = 4$.

ii. *2D Case:* Let $A \in \mathbb{R}^{2 \times 2}$ be, for instance,

$$A = \begin{pmatrix} 0.7 & 0.1 \\ 0.2 & 0.6 \end{pmatrix}, \quad B = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad C = \begin{pmatrix} 1 & 0 \end{pmatrix}, \quad D = 0$$

Compute kernels up to $L = 3$ and briefly discuss how the kernel captures the “impulse response”.

- (c) **Efficient Computation with Convolutions.** If we already know the kernel K , how much can we parallelize the computation of the output sequence $\{y_k\}$ for an input sequence $\{u_k\} \in \mathbb{R}^d$ of length L ? What is the minimum critical path length of the computation? What about a naive, direct computation of y_k from the unrolled recursion?
- (d) **Efficient Kernel Computation.** Given A, B, C , how can we compute the kernel, K , efficiently? What are some strategies to parallelize kernel computation? You may assume $L = 2^N$ for some N for simplicity.
- (e) **Adding structure to A .** Suppose A is a diagonal matrix. How can we leverage this structure to compute the kernel K more efficiently?
- (f) **Diagonal-plus-low-rank (DPLR) structure** Now if A has the following form:

$$A = I_n + pp^\top,$$

where $A \in \mathbb{R}^{n \times n}, p \in \mathbb{R}^n$. How can we leverage this structure to compute the kernel K more efficiently?

5. Coding SSM Forward

You showed that SSMs forward pass can be implemented using recurrent and convolution approaches in the previous problem. In this problem, we will implement the two approaches and compare their efficiency. Particularly, you are given the following update equation:

$$\mathbf{h}_{t+1} = (W \cdot \mathbf{h}_t + U \cdot \mathbf{x}_t + \mathbf{b})$$

You are given all the input $\mathbf{x}_t$ for $t = 1, 2, \dots, T$ and the initial hidden state $\mathbf{h}_0 = 0$. You need to implement the forward pass for the SSM computing all the hidden states $\mathbf{h}_t$ for $t = 1, 2, \dots, T$.

We will complete the notebooks `q_coding_ssm_forward_cpu.ipynb` and `q_coding_ssm_forward_gpu.ipynb` on the CPU and GPU respectively.

CPU Implementation:

- (a) Implement the forward pass by unrolling the SSM in time in a recurrent manner.
- (b) Implement the forward pass by using a convolution-based approach. You will create the kernel K and use `nn.Conv1d` to apply the kernel to the input x . Hint: You can create the kernel using a divide-and-conquer method.
- (c) Compare the runtime of the two approaches and explain your findings.

GPU Implementation:

- (d) Now implement the forward pass on the GPU. You can copy your existing implementation to the GPU notebook.
- (e) Compare the runtime of the two approaches and explain your findings.
- (f) To optimize the convolution-based approach, we will constrain W to be diagonal and leverage depth-wise convolutions. Implement the new forward pass under this constraint and now compare the run-times.

6. Homework Process and Study Group

Citing sources and collaborators are an important part of life, including being a student!

We also want to understand what resources you find helpful and how much time homework is taking, so we can change things in the future if possible.

(a) **What sources (if any) did you use as you worked through the homework?**

(b) **If you worked with someone on this homework, who did you work with?**

List names and student ID's. (In case of homework party, you can also just describe the group.)

(c) **Roughly how many total hours did you work on this homework?**

Contributors:

- Linyuan Gong.
- Olivia Watkins.
- Jerome Quenum.
- Anant Sahai.
- Anrui Gu.
- Matthew Lacayo.
- Past EECS 282 and 227 Staff.
- Kumar Krishna Agrawal.
- Sheng Shen.
- Naman Jain.
- Qiyang Li.